ODOO VIEWS, SECURITY, AND QWEB REPORTS

Views, Security & QWeb

UI views, access rights, and report basics

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026

WHAT WE WILL COVER

Agenda

1. Section 1: List and Search Views — 10 topics
2. Section 2: Form and Analytical Views — 8 topics
3. Section 3: View Inheritance Positions and Groups — 8 topics
4. Section 4: Access Rights, Record Rules, and Data — 11 topics
5. Section 5: Reports, Templates, and Automation — 9 topics

1 Section 1

List and Search Views

1.1 List View Basics

- A list view shows many records in rows.

Simple list view

```
<tree string="Courses">
    <field name="name"/>
    <field name="teacher_id"/>
# ...
```

- Choose columns that help users decide what to open.

List view action

```
<field name="view_mode">tree,form</field>
<field name="res_model">training.course</field>
```

1.2 Editable List

- Editable lists let users change rows without opening a form.

Edit at bottom

```
<tree editable="bottom">
    <field name="product_id"/>
    <field name="quantity"/>
# ...
```

- Use them for simple line data, not complex workflows.

Edit at top

```
<tree editable="top">
    <field name="name"/>
    <field name="score"/>
# ...
```

1.3 List Decorations

- Decorations color rows based on field values.

Danger row

```
<tree decoration-danger="state == 'late'">
    <field name="name"/>
    <field name="state"/>
# ...
```

- They guide attention but do not replace real validation.

Hidden helper field

```
<field name="deadline_passed" invisible="1"/>
<tree decoration-warning="deadline_passed">
    <field name="name"/>
# ...
```

1.4 List Button

- A list button starts an action from a row.

Object button

```
<button name="action_confirm"
        type="object"
        string="Confirm"
# ...
```

- Use object buttons for Python model methods.

Button with state rule

```
<button name="action_reset"
        type="object"
        string="Reset"
# ...
```

1.5 Sort and Limit

- default_order controls the first sort users see.

Newest first

```
<tree default_order="create_date desc" limit="80">
    <field name="name"/>
</tree>
```

- limit controls how many rows load at first.

Priority queue

```
<tree default_order="priority desc, date_deadline">
    <field name="priority"/>
    <field name="date_deadline"/>
# ...
```

1.6 Group By in Search

- Group by organizes records into collapsible buckets.

Group by teacher

```
<filter string="Teacher"
        name="group_teacher"
        context="{'group_by': 'teacher_id'}"/>
```

- Put common groupings in the search view.

Group by month

```
<filter string="Start Month"
        name="group_start_month"
        context="{'group_by': 'start_date:month'}"/>
```

1.7 Search View Basics

- A search view defines quick filters and search fields.

Search by fields

```
<search string="Courses">
    <field name="name"/>
    <field name="teacher_id"/>
# ...
```

- It supports both typed search and saved filter buttons.

Simple filter

```
<filter string="Published"
        name="published"
        domain="[('is_published', '=', True)]"/>
```

1.8 Search Panel

- The search panel gives a left-side filter menu.

Category panel

```
<searchpanel>
    <field name="category_id"/>
</searchpanel>
```

- It works well for categories, companies, and owners.

Multi-select panel

```
<searchpanel>
    <field name="tag_ids" select="multi"/>
</searchpanel>
```

1.9 Default Filter

- A default filter opens an action with a filter already active.

Action context

```
<field name="context">
    {'search_default_published': 1}
</field>
```

- Use the action context to enable it.

Matching filter

```
<filter string="Published"
        name="published"
        domain="[('is_published', '=', True)]"/>
```

1.10 Action View Modes

- view_mode controls which views users can switch between.

List first

```
<field name="view_mode">tree,form,kanban</field>
```

- Put the most common view first.

Report first

```
<field name="view_mode">pivot,graph,tree</field>
```

2 Section 2

Form and Analytical Views

2.1 Form Sheet and Group

- A sheet gives the form a document-like area.

Basic form

```
<form string="Course">
    <sheet>
        <group>
# ...
```

- Groups align labels and fields in readable columns.

Two columns

```
<group>
    <group><field name="start_date"/></group>
    <group><field name="end_date"/></group>
# ...
```

2.2 Notebook Pages

- A notebook splits a form into tabs.

Notebook with lines

```
<notebook>
    <page string="Students">
        <field name="student_ids"/>
# ...
```

- Use tabs for secondary details, not the main identity.

Notes tab

```
<page string="Notes">
    <field name="description"/>
</page>
```

2.3 Statusbar

- A statusbar shows the record stage.

State field

```
<field name="state"
       widget="statusbar"
       statusbar_visible="draft,confirmed,done"/>
```

- It usually uses a selection field named state.

Workflow button

```
<button name="action_confirm"
        type="object"
        string="Confirm"
# ...
```

2.4 Kanban Basics

- Kanban views show records as cards.

Small card

```
<kanban>
    <templates>
        <t t-name="kanban-box">
# ...
```

- They are useful for pipelines, tasks, and visual queues.

Group by stage

```
<kanban default_group_by="stage_id">
    <field name="stage_id"/>
</kanban>
```

2.5 Graph View

- A graph view turns records into charts.

Bar chart

```
<graph string="Enrollments" type="bar">
    <field name="teacher_id"/>
    <field name="student_count" type="measure"/>
# ...
```

- Use measures for numbers and grouping fields for axes.

Line chart

```
<graph string="Revenue" type="line">
    <field name="date" interval="month"/>
    <field name="amount_total" type="measure"/>
# ...
```

2.6 Pivot View

- A pivot view helps users summarize records.

Pivot by teacher

```
<pivot string="Course Analysis">
    <field name="teacher_id" type="row"/>
    <field name="student_count" type="measure"/>
# ...
```

- Rows and columns are dimensions.

Pivot by month

```
<pivot string="Monthly Sales">
    <field name="date_order" interval="month" type="col"/>
    <field name="amount_total" type="measure"/>
# ...
```

2.7 Invisible, Readonly, and Required

- View attributes guide user input in the client.

Conditional readonly

```
<field name="teacher_id"
       readonly="state != 'draft'"/>
```

- They improve usability but do not replace server rules.

Conditional required

```
<field name="cancel_reason"
       invisible="state != 'cancelled'"
       required="state == 'cancelled'"/>
```

2.8 View Inheritance XPath

- XPath lets one module modify another module's view.

Find a field

```
<xpath expr="//field[@name='name']" position="after">
    <field name="code"/>
</xpath>
```

- Use stable anchors such as field names or page names.

Find a page

```
<xpath expr="//page[@name='settings']" position="inside">
    <group string="Training"/>
</xpath>
```

3 Section 3

View Inheritance Positions and Groups

3.1 Position After

- position='after' inserts new XML after the matched node.

After a field

```
<xpath expr="//field[@name='name']" position="after">
    <field name="short_code"/>
</xpath>
```

- It is common for adding a field beside an existing field.

After a button

```
<xpath expr="//button[@name='action_confirm']" position="after">
    <button name="action_print" type="object" string="Print"/>
</xpath>
```

3.2 Position Before

- position='before' inserts new XML before the matched node.

Before date

```
<xpath expr="//field[@name='start_date']" position="before">
    <field name="priority"/>
</xpath>
```

- It is useful when the new field should be seen first.

Before page

```
<xpath expr="//page[@name='notes']" position="before">
    <page name="students" string="Students"/>
</xpath>
```

3.3 Position Inside

- position='inside' adds content inside a matched container.

Inside group

```
<xpath expr="//group[@name='main']" position="inside">
    <field name="level"/>
</xpath>
```

- It works with group, page, sheet, and header nodes.

Inside header

```
<xpath expr="//header" position="inside">
    <button name="action_archive" type="object" string="Archive"/>
</xpath>
```

3.4 Position Replace

- position='replace' removes the matched node and inserts new XML.

Replace a field

```
<xpath expr="//field[@name='description']" position="replace">
    <field name="description" placeholder="Write notes here"/>
</xpath>
```

- Use it carefully because it can break later inherited views.

Remove by empty replace

```
<xpath expr="//field[@name='legacy_code']" position="replace"/>
```

3.5 Security Groups

- A group represents a security role.

Teacher group

```
<record id="group_training_teacher" model="res.groups">
    <field name="name">Teacher</field>
</record>
```

- Users receive permissions through their groups.

Student group

```
<record id="group_training_student" model="res.groups">
    <field name="name">Student</field>
</record>
```

3.6 Group Category

- A category groups related security roles in the user form.

Category record

```
<record id="module_category_training" model="ir.module.category">
    <field name="name">Training</field>
    <field name="sequence">30</field>
# ...
```

- It makes permissions easier to understand.

Group in category

```
<field name="category_id"
       ref="module_category_training"/>
```

3.7 Implied Groups

- implied_ids automatically adds another group.

Manager implies teacher

```
<field name="implied_ids"
       eval="[(4, ref('group_training_teacher'))]"/>
```

- Use it for role inheritance such as manager includes user.

Base user implied

```
<field name="implied_ids"
       eval="[(4, ref('base.group_user'))]"/>
```

3.8 Groups on Menu

- The groups attribute hides a menu from other users.

Teacher menu

```
<menuitem id="training_menu_teacher"
          name="Teacher Tools"
          groups="training.group_training_teacher"/>
```

- It is a usability rule, not a full security rule.

Manager menu

```
<menuitem id="training_menu_reports"
          name="Reports"
          groups="training.group_training_manager"/>
```

4 Section 4

Access Rights, Record Rules, and Data

4.1 ir.model.access.csv

- CSV access files grant model-level permissions.

CSV header

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
```

- They control read, write, create, and delete.

Teacher access row

```
access_course_teacher,course teacher,model_training_course,training.group_training_teacher,1,1,1,0
```

4.2 Access Rights XML

- Access rights can also be written as XML records.

XML access record

```
<record id="access_course_teacher_xml" model="ir.model.access">
    <field name="name">course teacher</field>
    <field name="model_id" ref="model_training_course"/>
# ...
```

- XML is useful when values need references or conditions.

No delete access

```
<field name="perm_unlink" eval="0"/>
<field name="perm_write" eval="1"/>
```

4.3 Record Rules

- Record rules filter which records a user can access.

Own records only

```
<field name="domain_force">
    [('user_id', '=', user.id)]
</field>
```

- They apply after model access rights.

Company records only

```
<field name="domain_force">
    [('company_id', 'in', company_ids)]
</field>
```

4.4 Student Record Rule Example

- A student should usually see only their own enrollments.

Student enrollment rule

```
<record id="rule_student_own_enrollments" model="ir.rule">
    <field name="name">Students see own enrollments</field>
    <field name="model_id" ref="model_training_enrollment"/>
# ...
```

- The rule links records to the current user's partner.

Model access row

```
access_enrollment_student,enrollment student,model_training_enrollment,training.group_training_student,1,0,0,0
```

4.5 Teacher Record Rule Example

- A teacher may see courses they teach.

Teacher course rule

```
<record id="rule_teacher_own_courses" model="ir.rule">
    <field name="name">Teachers see own courses</field>
    <field name="model_id" ref="model_training_course"/>
# ...
```

- The rule should match the business relationship.

Manager bypass group

```
<field name="groups"
       eval="[(4, ref('group_training_manager'))]"/>
```

4.6 Groups on Actions

- Groups on an action limit who can launch it.

Action with group

```
<record id="action_training_report" model="ir.actions.act_window">
    <field name="name">Training Report</field>
    <field name="groups_id" eval="[(4, ref('group_training_manager'))]"/>
# ...
```

- They help hide reports or tools from unrelated roles.

Server action group

```
<field name="groups_id"
       eval="[(4, ref('base.group_system'))]"/>
```

4.7 Groups on Fields

- A field can be visible only to selected groups.

View field group

```
<field name="internal_note"
       groups="training.group_training_manager"/>
```

- Use this for sensitive or advanced fields.

Python field group

```
internal_note = fields.Text(
    groups="training.group_training_manager"
)
```

4.8 XML Data noupdate

- noupdate protects data from being overwritten on module update.

Protected data

```
<data noupdate="1">
    <record id="seq_training_course" model="ir.sequence">
        <field name="name">Course Sequence</field>
# ...
```

- Use it for user-edited configuration records.

Normal data

```
<data>
    <record id="view_training_course_form" model="ir.ui.view"/>
</data>
```

4.9 CSV vs XML Data

- CSV is compact for many similar rows.

CSV access row

```
access_course_user,course user,model_training_course,base.group_user,1,0,0,0
```

- XML is better for nested records and references.

XML menu record

```
<menuitem id="training_menu_root"
          name="Training"
          sequence="20"/>
```

4.10 Sequence Basics

- ir.sequence generates readable document numbers.

Sequence XML

```
<record id="seq_training_course" model="ir.sequence">
    <field name="name">Course</field>
    <field name="code">training.course</field>
# ...
```

- Use a code that your model can call.

Python use

```
vals["name"] = self.env["ir.sequence"].next_by_code(
    "training.course"
)
```

4.11 External IDs

- External IDs let XML and CSV records refer to each other.

Reference a group

```
<field name="group_id" ref="training.group_training_teacher"/>
```

- They must stay stable across upgrades.

Reference a model

```
<field name="model_id" ref="model_training_course"/>
```

5 Section 5

Reports, Templates, and Automation

5.1 QWeb Report Basics

- QWeb reports render HTML that can become PDF.

Report action

```
<record id="action_report_course" model="ir.actions.report">
    <field name="name">Course Report</field>
    <field name="model">training.course</field>
# ...
```

- A report action connects the model to the template.

Template shell

```
<template id="report_course">
    <t t-call="web.html_container">
        <t t-foreach="docs" t-as="doc"/>
# ...
```

5.2 t-field and t-out

- t-field renders an Odoo field with formatting.

Formatted field

```
<span t-field="doc.start_date"/>
```

- t-out prints a computed expression safely.

Expression output

```
<span t-out="doc.name.upper()"/>
```

5.3 QWeb If

- t-if conditionally shows a piece of template.

Optional note

```
<p t-if="doc.note">
    <span t-field="doc.note"/>
</p>
```

- Use it for optional values and simple branches.

State message

```
<strong t-if="doc.state == 'done'">Completed</strong>
```

5.4 QWeb Foreach

- t-foreach repeats markup for each record or value.

Loop lines

```
<tr t-foreach="doc.line_ids" t-as="line">
    <td><span t-field="line.name"/></td>
</tr>
```

- Use t-as to name the current item.

Loop numbers

```
<li t-foreach="[1, 2, 3]" t-as="number">
    <span t-out="number"/>
</li>
```

5.5 Paper Format

- Paper formats define page size and margins.

A4 format

```
<record id="paperformat_training" model="report.paperformat">
    <field name="name">Training A4</field>
    <field name="format">A4</field>
# ...
```

- Attach one when a report needs special layout.

Attach to report

```
<field name="paperformat_id"
       ref="paperformat_training"/>
```

5.6 Email Template

- Email templates generate messages from records.

Template record

```
<record id="email_course_confirmed" model="mail.template">
    <field name="name">Course Confirmed</field>
    <field name="model_id" ref="model_training_course"/>
# ...
```

- Use placeholders for record-specific values.

Body expression

```
<field name="body_html" type="html">
    <p>Hello <t t-out="object.teacher_id.name"/>.</p>
</field>
```

5.7 Server Action

- A server action runs server-side logic from the UI or automation.

Call Python method

```
<record id="server_action_confirm_courses" model="ir.actions.server">
    <field name="name">Confirm Courses</field>
    <field name="model_id" ref="model_training_course"/>
# ...
```

- Use it for small administrative actions.

Bind to model

```
<field name="binding_model_id" ref="model_training_course"/>
<field name="binding_view_types">list,form</field>
```

5.8 Header Buttons and Security

- Header buttons should match the record workflow.

Manager-only button

```
<button name="action_approve"
        type="object"
        string="Approve"
# ...
```

- Use groups to hide actions from users who cannot run them.

State-based button

```
<button name="action_done"
        type="object"
        string="Done"
# ...
```

5.9 Manifest Data Order

- Data files load in the order listed in the manifest.

Manifest order

```
"data": [
    "security/groups.xml",
    "security/ir.model.access.csv",
# ...
```

- Load security before views that use security groups.

Report files

```
"data": [
    "report/course_templates.xml",
    "report/course_reports.xml",
# ...
```

NEXT STEP

Try the examples on your machine

Then open the next module deck

Merit Advisory
05 · Views & Security